

Documentation Infrastructure

Introduction

Dans le cadre du Workshop national Bac+4 « Mission SENTINEL-X », notre groupe (G7) s'est constitué en consortium d'ingénieurs chargé de concevoir un boîtier de surveillance autonome pour les microcentrales énergétiques d'Aether Corp Industrial Solutions, déployées en zones isolées et exposées à des menaces à la fois cyber, physiques et environnementales.

Ce document présente la partie Infrastructure du projet

Prérequis et infra

Pré-requis de l'infrastructure :

- Bases de données
- Cluster mosquitto
- Serveur API dev pour le dashboard
- Serveur IA pour la gestion du flux vidéo
- Serveur de déploiement et d'automatisation
- Serveur de monitoring

Infrastructure mise en place :

- Cluster Galera 3 noeuds
- Cluster Mosquitto
- Serveur API dev
- Serveur pour héberger l'IA Sentinelle et Yolo
- Serveur Ansible
- Serveur Zabbix

Mise en place du cluster Galera

Pour assurer la haute disponibilité des données, nous avons mis en place un cluster MariaDB Galera composé de 3 nœuds. Galera utilise une réplication synchrone et un système de maître et réplication donc nous avons dans celle-ci :

- 1 Master
- 2 Réplica

Nommées ainsi :

- **WORKSHOP-GRP7-BDD1** (master)
- **WORKSHOP-GRP7-BDD2** (réplica 1)
- **WORKSHOP-GRP7-BDD3** (réplica 2)

Chacun de ces nœuds héberge une copie complète de la base de données et dispose de la même configuration :

- Conteneur LXC (Linux)
- CPU : 2
- RAM : 2 Go
- Espace disque 50 Go

Les données sont synchronisées automatiquement entre les trois serveurs.

Un nombre impair de nœuds permet au cluster de conserver un « quorum » (une majorité) en cas de panne. Avec 3 nœuds, la perte d'un serveur n'interrompt donc pas le service, ce qui n'aurait pas été le cas avec seulement 2 nœuds.

Bénéfices obtenus :

- **Haute disponibilité** : le service reste accessible en cas de panne d'un serveur
- **Intégrité des données** : les 3 nœuds contiennent toujours les mêmes informations
- **Maintenance facilitée** : un nœud peut être arrêté ou mis à jour sans coupure
- **Évolutivité** : l'ajout d'un nœud supplémentaire reste possible

Cluster Mosquitto MQTT

Pour assurer la remontée des données des capteurs vers la stack serveur, nous avons mis en place un broker Mosquitto MQTT. MQTT est un protocole de messagerie léger, fonctionnant sur le principe de publication / abonnement, l'ESP8266 publie les mesures (température, humidité, gaz, présence) sur des « topics », et les services concernés (API, base de données, dashboard) s'abonnent pour les recevoir en temps réel. Ce protocole est particulièrement adapté à l'IoT car il consomme très peu de ressources et supporte bien les connexions instables.

Rôle du broker dans l'architecture :

- Publieur : l'ESP8266 (capteurs DHT22, MQ-2, PIR)
- Broker : Mosquitto, hébergé dans un conteneur Docker sur le PC Serveur Local
- Abonnés : l'API et le dashboard de supervision

Nommé : WORKSHOP-GRP7-BrokerMosquitto

Phase 1 : tests en environnement local (port 1883)

Dans un premier temps, nous avons déployé Mosquitto en environnement local, sur le port standard 1883 (MQTT non chiffré). Cette étape nous a permis de :

- Valider le bon fonctionnement du broker et de son conteneur
- Tester la communication entre l'ESP8266 et le serveur
- Vérifier la publication et la réception des messages sur les différents topics
- Sécuriser l'intégration avec les équipes DevOps avant d'ajouter la couche de chiffrement

Cette configuration était volontairement limitée à des tests, car les données transitent en clair sur le réseau.

Phase 2 : sécurisation par certificat TLS auto-signé

Une fois la chaîne de communication validée, nous avons renforcé la sécurité en mettant en place le chiffrement TLS (MQTTS). Un certificat auto-signé a été généré et configuré sur le broker, ce qui permet de :

- Chiffrer les échanges entre l'ESP8266 et Mosquitto de bout en bout
- Empêcher l'interception ou la lecture des données (attaques de type « homme du milieu »)
- Répondre à l'exigence de chiffrement des flux du cahier des charges

Le certificat étant auto-signé, il n'est pas validé par une autorité de certification publique. Ce choix est cohérent pour un réseau local isolé, à condition que l'ESP8266 embarque le certificat de l'autorité utilisée pour vérifier l'identité du broker.

Bénéfices obtenus :

- **Légèreté** : protocole économe en ressources, adapté à l'ESP8266
- **Temps réel** : les données sont transmises instantanément au dashboard
- **Découplage** : les capteurs et les services consommateurs sont indépendants les uns des autres
- **Sécurité** : flux chiffrés par TLS
- **Portabilité** : déploiement simple et reproductible grâce à Docker-Compose

Serveurs API web et IA Yolo | Sentinel

Pour permettre aux équipes DevOps de déployer et configurer leurs environnements, nous avons réparti les conteneurs Docker sur deux machines virtuelles distinctes. Cette séparation isole les services entre eux et évite qu'un incident sur une VM n'affecte l'ensemble de la stack.

L'accès aux VM se fait exclusivement par SSH sur le port 22, avec une authentification par clé publique. Aucune connexion par mot de passe n'est autorisée, conformément aux exigences de durcissement du cahier des charges.

Le fonctionnement est le suivant :

- Chaque DevOps génère sa propre paire de clés (privée / publique)
- Sa clé publique est renseignée dans le fichier `authorized_keys` de la VM
- Sa clé privée reste sur son poste et n'est jamais partagée

Chacun peut ainsi se connecter avec son identité propre, sans avoir à partager de mot de passe ni de secret commun.

Nommés :

- WORKSHOP-GRP7-APIDev
- WORKSHOP-GRP7-YOLO

Une fois celles-ci créées, les DevOps pouvaient ensuite créer et insérer le code de leur IA ainsi que du dashboard.

Serveur Ansible

Pour automatiser le déploiement de l'infrastructure, nous avons mis en place un serveur Ansible. Ansible est un outil d'automatisation qui permet de configurer plusieurs machines à partir d'un point central, sans installer d'agent sur les machines cibles, il s'y connecte en SSH et applique des scripts de configuration appelés « playbooks ».

Nommé : **WORKSHOP-GRP7-Ansible**

Le serveur Ansible a été utilisé pour deux déploiements :

- le cluster Galera (WORKSHOP-GRP7-BDD1, BDD2 et BDD3)

- le serveur Zabbix, ainsi que les agents Zabbix sur toutes les machines de l'infrastructure

Le serveur Ansible s'appuie sur la même logique d'accès que le reste de l'infrastructure : connexion SSH par clé publique vers les machines à configurer.

Le serveur Zabbix assure la supervision de l'ensemble de l'infrastructure. Chaque machine remonte ses indicateurs (CPU, RAM, disque, état des services) via son agent Zabbix, ce qui permet de suivre l'état du parc depuis un seul tableau de bord et de répondre à l'exigence de monitoring et de maintien en condition opérationnelle (MCO) du cahier des charges.

Bénéfices obtenus :

- **Gain de temps** : un même playbook configure plusieurs machines en une seule exécution
- **Homogénéité** : toutes les machines sont configurées de la même manière, sans oubli ni erreur manuelle
- **Reproductibilité** : l'infrastructure peut être redéployée rapidement en cas de problème
- **Supervision complète** : chaque machine est surveillée grâce à son agent Zabbix
- **Évolutivité** : l'ajout d'une nouvelle machine se fait simplement en l'intégrant à l'inventaire Ansible

Serveur Zabbix

Pour assurer le suivi de l'infrastructure, nous avons mis en place un serveur Zabbix, outil de supervision qui collecte en continu l'état des machines et des services. Son déploiement, ainsi que celui des agents, a été réalisé via Ansible (voir la partie précédente).

Une fois les agents installés sur les machines, il ne restait qu'à les déclarer côté Zabbix : nous avons ajouté dans nos collections (groupes d'hôtes) les différentes VM sur lesquelles les agents avaient été déployés. Zabbix a alors pu interroger chaque agent et remonter ses données.

Nommé : **WORKSHOP-GRP7-Zabbix**

Grâce à cette étape, l'ensemble de notre infrastructure est désormais monitorée depuis un seul point :

- les machines hébergeant le cluster Galera (WORKSHOP-GRP7-BDD1, BDD2 et BDD3)
- les VM hébergeant les conteneurs Docker (dont le broker Mosquitto)
- le serveur Ansible

Pour chacune d'elles, Zabbix suit notamment la consommation CPU, la mémoire, l'espace disque et la disponibilité de la machine.

Bénéfices obtenus :

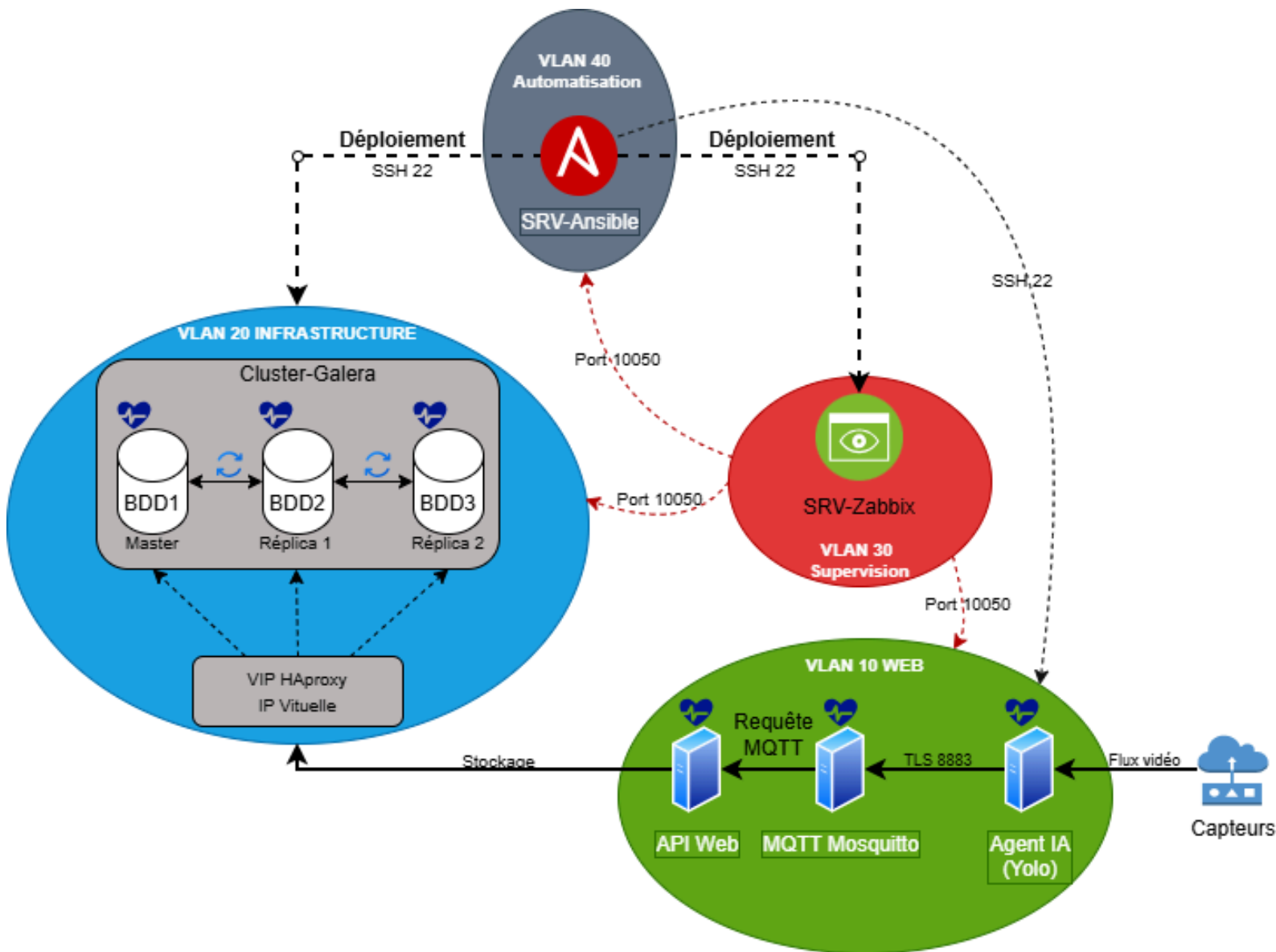
- **Vision globale** : l'état de toute l'infrastructure est visible depuis une seule interface

- Détection rapide : une machine indisponible ou surchargée est repérée immédiatement
- **Maintien en condition opérationnelle** : la surveillance du CPU, de la RAM et du disque répond à l'exigence de MCO du cahier des charges
- **Mise en place rapide** : grâce au déploiement automatisé des agents par Ansible, l'ajout des machines dans Zabbix s'est fait simplement
- **Évolutivité** : une nouvelle machine peut être supervisée en déployant un agent puis en l'ajoutant à la collection

Vision global du zabbix :

☐ Name ▲	Items	Triggers	Graphs	Discovery	Web	Interface	Proxy	Templates	Status	Availability	Agent encryption	Info	Tags
☐ WORKSHOP-GRP7-APIDev	Items 59	Triggers 24	Graphs 11	Discovery 3	Web	172.16.137.5:10050		Linux by Zabbix agent	Enabled	ZBX	None		
☐ WORKSHOP-GRP7-BDD1	Items 59	Triggers 24	Graphs 11	Discovery 3	Web	172.16.137.1:10050		Linux by Zabbix agent	Enabled	ZBX	None		
☐ WORKSHOP-GRP7-BDD2	Items 59	Triggers 24	Graphs 11	Discovery 3	Web	172.16.137.2:10050		Linux by Zabbix agent	Enabled	ZBX	None		
☐ WORKSHOP-GRP7-BDD3	Items 59	Triggers 24	Graphs 11	Discovery 3	Web	172.16.137.3:10050		Linux by Zabbix agent	Enabled	ZBX	None		
☐ WORKSHOP-GRP7-BrockerMosquito	Items 59	Triggers 24	Graphs 11	Discovery 3	Web	172.16.137.4:10050		Linux by Zabbix agent	Enabled	ZBX	None		
☐ WORKSHOP-GRP7-YOLO	Items 59	Triggers 24	Graphs 11	Discovery 3	Web	172.16.137.6:10050		Linux by Zabbix agent	Enabled	ZBX	None		
☐ Zabbix server	Items 146	Triggers 78	Graphs 14	Discovery 6	Web	127.0.0.1:10050		Linux by Zabbix agent, Zabbix server health	Enabled	ZBX	None		

Infrastructure complète :



Légende :

VLAN 10

VLAN 20

VLAN 30

VLAN 40

Agent Zabbix

Description de l'infrastructure

L'infrastructure du projet SENTINEL-X est segmentée en quatre VLAN, chacun regroupant des machines selon leur fonction. Cette segmentation isole les services entre eux, limite la propagation d'une éventuelle compromission et permet de contrôler précisément les flux autorisés.

VLAN	Nom	Rôle	Machines
VLAN 10	WEB	Collecte, traitement et exposition des données	API Web, MQTT Mosquitto, Agent IA (YOLO)
VLAN 20	Infrastructure	Stockage des données en haute disponibilité	Cluster Galera (BDD1, BDD2, BDD3) et VIP HAProxy
VLAN 30	Supervision	Monitoring de l'ensemble du parc	SRV-Zabbix
VLAN 40	Automatisation	Déploiement et configuration automatisés	SRV-Ansible

VLAN 10 : WEB

Ce VLAN porte la chaîne applicative de bout en bout :

- Agent IA (YOLO) : reçoit le flux vidéo des capteurs, l'analyse en temps réel et détecte une présence suspecte.
- MQTT Mosquitto : broker de messagerie qui reçoit les données de l'Agent IA. Les échanges sont chiffrés en TLS sur le port 8883 (MQTTS).
- API Web : s'abonne au broker par requêtes MQTT, récupère les données et les met à disposition du dashboard de supervision.

VLAN 20 : Infrastructure

Ce VLAN héberge le cluster Galera de trois nœuds :

- BDD1 : master
- BDD2 : réplica 1
- BDD3 : réplica 2

Les trois nœuds sont synchronisés en permanence. Devant le cluster se trouve une VIP HAProxy (adresse IP virtuelle) : l'API Web ne s'adresse jamais directement à un nœud, mais à cette adresse unique. HAProxy répartit ensuite les requêtes vers les nœuds disponibles.

En cas de panne d'un serveur, le service continue donc sans modification côté applicatif.

VLAN 30 : Supervision

Le serveur SRV-Zabbix surveille toute l'infrastructure. Des agents Zabbix sont installés sur les machines (symbolisés par le cœur bleu sur le schéma) : les trois nœuds Galera, l'API

Web, Mosquitto et l'Agent IA. Zabbix interroge ces agents sur le port 10050 et centralise leurs indicateurs (CPU, RAM, disque, disponibilité).

VLAN 40 : Automatisation

Le serveur SRV-Ansible déploie et configure les machines à distance en SSH (port 22). Il a permis de mettre en place :

- le cluster Galera dans le VLAN 20
- le serveur Zabbix dans le VLAN 30
- les agents Zabbix, ainsi que la configuration des machines du VLAN 10

Flux de données

Le parcours d'une donnée, des capteurs jusqu'au stockage, est le suivant :

1. Les capteurs envoient leur flux vidéo à l'Agent IA (YOLO).
2. L'Agent IA publie ses résultats sur Mosquitto, en TLS sur le port 8883.
3. L'API Web récupère les messages par requête MQTT.
4. L'API enregistre les données dans le cluster, via la VIP HAProxy.

7. Matrice des flux réseau

Source	Destination	Protocole / Port	Usage
SRV-Ansible (VLAN 40)	Cluster Galera (VLAN 20)	SSH 22	Déploiement du cluster
SRV-Ansible (VLAN 40)	SRV-Zabbix (VLAN 30)	SSH 22	Déploiement du serveur de supervision
SRV-Ansible (VLAN 40)	Machines du VLAN 10	SSH 22	Déploiement des agents et configuration
SRV-Zabbix (VLAN 30)	Machines des VLAN 10, 20 et 40	TCP 10050	Collecte des métriques des agents
Agent IA	MQTT Mosquitto	TLS 8883	Publication chiffrée des données
MQTT Mosquitto	API Web	MQTT	Transmission des messages
API Web	VIP HAProxy (VLAN 20)	SQL (3306)	Stockage des données

Points forts de l'architecture

- **Segmentation** : chaque fonction est isolée dans son propre VLAN, ce qui réduit la surface d'attaque.
- **Haute disponibilité** : le cluster Galera et la VIP HAProxy suppriment le point de défaillance unique sur la base de données.
- **Sécurité des flux** : les données sont chiffrées par TLS et les accès d'administration se font par SSH avec clés publiques.
- **Automatisation** : Ansible rend le déploiement reproductible et homogène.

- **Supervision** : l'ensemble du parc est surveillé depuis une seule interface.

Conclusion

La mise en place de cette infrastructure nous a permis de construire un socle technique complet, sécurisé et résilient pour le projet SENTINEL-X. Chaque besoin du cahier des charges a trouvé sa réponse : une base de données en haute disponibilité grâce au cluster Galera et à la VIP HAProxy, une remontée des données en temps réel via le broker Mosquitto, des serveurs dédiés à l'API et à l'IA pour les équipes DevOps, ainsi qu'un déploiement automatisé et une supervision centralisée grâce à Ansible et Zabbix.

La segmentation en quatre VLAN, associée au chiffrement des flux en TLS et aux accès SSH par clés publiques, nous a permis de limiter la surface d'attaque et de répondre aux exigences de sécurité du projet. L'automatisation avec Ansible a rendu le déploiement reproductible et homogène, tandis que Zabbix assure le maintien en condition opérationnelle de l'ensemble du parc depuis une seule interface.

Ce travail nous a également permis de mieux comprendre la logique d'une infrastructure complète, de la conception du schéma réseau jusqu'à la supervision, en passant par la sécurisation des flux et l'automatisation des déploiements. La collaboration avec les équipes DevOps a été déterminante pour que chaque partie du travail puisse s'accorder.

Plusieurs pistes d'amélioration pourraient être envisagées à l'avenir :

- mettre en place des sauvegardes automatisées de la base de données
- passer d'un broker unique à un véritable cluster Mosquitto, pour supprimer ce point de défaillance
- configurer des alertes Zabbix (e-mail, messagerie) pour être prévenu en cas d'incident
- renforcer le hardening des machines et l'authentification sur le broker